Український Католицький Університет

Курс «Штучний інтелект» · Семестровий проєкт · 2026

Applications of Classical ML for assisting Quantum Computing / QML on NISQ era quantum computers

Використання ML для зменшення рівня шуму в квантових програмах та алгоритмах

Команда:

Куриляк Данило

Сольчаник Василь

Ментор:

Сумик Марта

Пропозиція проєкту, подана в межах курсу «Штучний інтелект»

Факультет прикладних наук

2026-09-20

1. Проблема та мотивація

Квантові комп'ютери NISQ-ери дуже зашумлені: кожна операція вносить похибку, і сумарна помилка зростає з глибиною схеми. Похибку можна записати як

$$\varepsilon = |\langle O \rangle_{ideal} - \langle O \rangle_{noisy}|,$$

де $\langle O \rangle_{ideal}$ - ідеальний результат вимірювання, а $\langle O \rangle_{noisy}$ - те, що реально видає пристрій. Мета error mitigation, це зменшити ε лише за рахунок обробки результатів, без покращення заліза. Наявні методи спираються на прості евристичні правила, які погано пристосовуються до різних типів схем і шумів. Ми хочемо перевірити, чи зможе модель машинного навчання вивчити ці закономірності краще. Навіть невелике покращення точності може відчутно розширити коло задач, які реально розв'язувати на сьогоднішніх квантових комп'ютерах.

2. Огляд джерел та наявних рішень

Найпоширеніший підхід до error mitigation, це zero-noise extrapolation (ZNE): схему запускають кілька разів при штучно збільшеному рівні шуму $\lambda_1, \ldots, \lambda_k$, а потім екстраполюють результат до нульового шуму:

$$\langle O \rangle_{mitigated} \approx \lim_{\lambda \to 0} f(\lambda).$$

Метод не потребує додаткового квантового заліза [1], але вимагає багато повторних запусків, тому стає повільним на великих і глибоких схемах.

Тому дослідники пробують замінити такі ручні правила моделями машинного навчання. Liao et al. навчили кілька моделей, від лінійної регресії до графових нейромереж, передбачати "чистий" результат схеми напряму, без повторних запусків, і отримали точність на рівні ZNE при значно нижчій вартості обчислень [2]. Bennewitz et al. тренували модель на простих схемах, які можна порахувати класично, і застосовували її до складніших [3].

Формально такий ML-підхід передбачає поправку Δ: $\langle O \rangle_{mitigated} = \langle O \rangle_{noisy} + \Delta_\theta(x)$, де θ, це параметри моделі, а x, це вектор характеристик схеми та шуму. Саме на цій ідеї плануємо базувати свій підхід.

3. Дані

Для навчання моделі плануємо переважно синтетичні дані, згенеровані через Qiskit Aer [4], який моделює квантові схеми з різними noise models на основі характеристик реальних IBM QPU. Додатково використаємо calibration data IBM Quantum (T_1, T_2, gate error, readout error) через офіційний API [5].

Розподіл даних:

- **80%** - synthetic training data: базові схеми з різними типами й рівнями шуму (Qiskit Aer).
- **10%** - synthetic validation/test data: окрема частина синтетичних схем.
- **10%** - real-world test data: результати з IBM Quantum hardware через API.

Це дозволяє перевірити не лише узагальнення в межах синтетичних даних, а й перенесення моделі на реальне залізо.

Qiskit Aer та Qiskit IBM Runtime поширюються за ліцензією Apache 2.0 [4, 6], тому їх використання для генерації даних юридичних обмежень не створює. Сама ж calibration data IBM не підпадає під Apache 2.0: ліцензія стосується лише коду Qiskit, а умови використання даних з API визначає IBM Quantum End User Agreement [7]. Відповідно, у заявці зазначено:

> *License: Synthetic data will be generated using Qiskit Aer (Apache License 2.0). Real-world calibration and backend data will be obtained directly from IBM Quantum through its official API and used in accordance with the applicable IBM Quantum End User Agreement.*

4. Підхід і методи

Задачу формулюємо як регресію: знайти функцію $f_\theta : R^d \to R$, яка мінімізує похибку між передбаченою та справжньою поправкою на n прикладах:

$$\theta^* = \arg\min_\theta \frac{1}{n} \sum_{i=1}^{n} \left(f_\theta(x_i) - \Delta_i\right)^2,$$

де x_i - вектор характеристик i-ї схеми та шуму, а $\Delta_i = \langle O \rangle_{ideal,i} - \langle O \rangle_{noisy,i}$.

Не прив'язуємось до конкретної архітектури, а плануємо порівняти кілька моделей зі scikit-learn, XGBoost та PyTorch:

- **Baseline**: лінійна регресія - перевіряє, чи є в даних лінійно вловимий сигнал.
- **Градієнтний бустинг** (XGBoost / LightGBM): очікувано кращий баланс якості й швидкості.
- **Невелика нейромережа**: перевірка приросту точності проти вартості інференсу.

Моделі порівнюватимемо за точністю передбачення Δ (MSE/MAE) та швидкістю інференсу, оскільки compensation не має бути повільнішою за сам запуск схеми. Мета - перевершити евристичний baseline (ZNE) хоча б за одним критерієм, зберігши прийнятну точність.

5. Оцінювання результатів

Основна метрика, це середня абсолютна похибка (MAE) на тестовому бенчмарку схем:

$$MAE = \frac{1}{m} \sum_{j=1}^{m} \left| \langle O \rangle_{mitigated,j} - \langle O \rangle_{ideal,j} \right|.$$

Порівнюватимемо модель з евристичним бейзлайном Haiqu за цією метрикою, а також за швидкістю (час інференсу на одну схему), бо це важливо для продакшину.

6. План і ризики

До midterm: отримати й проаналізувати дані, побудувати перший baseline. До фінального захисту: натренувати й порівняти складніші моделі, обрати найкращу за MAE та швидкістю, оформити результати. Головний ризик, це недостатня кількість або якість даних про шум від Haiqu. План Б, генерувати додаткові синтетичні дані через симуляцію шуму, якщо реальних забракне.

7. Contribution statement

- **Куриляк Данило** — дані, препроцесинг, EDA
- **Сольчаник Василь** — моделювання, евалюація, документація

Список джерел

[1] Tudor Giurgica-Tiron, Yousef Hindy, Ryan LaRose, Andrea Mari, and William J. Zeng. Digital zero noise extrapolation for quantum error mitigation. *arXiv:2005.10921*, 2020.

[2] Haoran Liao, Derek S. Wang, Iskandar Sitdikov, Carlos Salcedo, Alireza Seif, and Zlatko K. Minev. Machine learning for practical quantum error mitigation. *arXiv:2309.17368*, 2023.

[3] Elizabeth R. Bennewitz, Florian Hopfmueller, Bohdan Kulchytskyy, Juan Carrasquilla, and Pooya Ronagh. Neural error mitigation of near-term quantum simulations. *Nature Machine Intelligence*, 4:618–624, 2022.

[4] Qiskit contributors. Qiskit: An open-source framework for quantum computing. Zenodo, 2024. doi:10.5281/zenodo.2573505.

[5] IBM Quantum. Retrieve backend properties and calibration data. IBM Quantum Documentation, 2024. URL: https://docs.quantum.ibm.com.

[6] IBM Quantum. Qiskit ibm runtime documentation. IBM Quantum Documentation, 2024. URL: https://docs.quantum.ibm.com/api/qiskit-ibm-runtime.

[7] IBM. Ibm quantum platform terms of use. IBM, 2024. URL: https://quantum.ibm.com/terms.